

Introduction to LangGraph

Why Use LangGraph?

Welcome to **Introduction to LangGraph**. We have built chains that process input in a fixed sequence and agents that dynamically select tools. Both are powerful — but what happens when a task requires steps that share data, branch based on results, loop until a condition is met, or pause for human approval?

LangGraph is a framework built on LangChain to create stateful, multi-step workflows as directed graphs. Each step is a node, the connections between steps are edges, and a shared state carries data throughout the workflow.

When Chains and Agents Fall Short

A chain follows a fixed path: prompt → model → parser. It cannot skip, repeat, or change direction based on intermediate results. An agent is more flexible — it reasons about which tool to call — but makes all decisions inside a single loop, with no way to enforce a specific order across stages or insert a human approval step.

LangGraph closes this gap by providing explicit control over workflow structure while maintaining the flexibility to branch, loop, and pause.

Thinking in Graphs

A graph has three components:

- **Nodes** — Python functions that read from the shared state, do some work, and write results back.
- **Edges** — Connections that define execution order. A simple edge runs one node after another; a conditional edge checks the state to decide which node runs next.
- **State** — A shared data structure (a TypedDict) that every node can read from and write to.

Core Components of LangGraph

- StateGraph is the main class. We initialize it with a state schema — a TypedDict that specifies the names and types of all shared fields. After creation, we register nodes and edges on it.

- `START` and `END` are special constants marking where the graph begins and ends. We connect `START` to the first node and the last node to `END`.
- `add_node("name", function)` registers a step. The function receives the current state and returns a dictionary with only the fields it updates. `LangGraph` automatically merges these updates into the shared state.
- `add_edge(source, target)` sets a fixed connection between two nodes, so when the source completes, the target runs next.
- `compile()` checks the graph and produces an executable object. We run it with `invoke()`, passing an initial state dictionary.

What We Will Build

Over the coming exercises, we will develop a content publishing pipeline — adding richer state, conditional routing, loops, human approval, and persistent memory. Each exercise introduces a new capability while the core pattern stays the same: nodes, edges, and state.

Instructions

For each scenario, decide whether it fits `LangChain` (chains/agents) or requires `LangGraph`, then drag it to the appropriate category.

Your First Graph

We understand what nodes, edges, and states are — now we build something with them. Here, we construct a **content publishing pipeline**: a two-step graph that takes a topic, drafts a blog post, and formats it for publication.

Defining the State Schema

Before building nodes or edges, we define the state — the shared data structure that every node can read from and write to. The state is a Python class that inherits from `TypedDict`, with each field having a name and a type.

For our pipeline, we need three string fields: `topic` (the input), `draft` (generated by the first node), and `final_content` (produced by the second node). When the pipeline starts, we provide the topic and leave the other fields empty. As the graph runs, each node populates its corresponding field.

Creating Node Functions

Each node receives the current state and returns a dictionary with only the fields it updates. LangGraph merges those updates into the shared state automatically.

- `draft_content` reads `topic`, prompts the language model for a short blog post, and returns the result under `draft`.
- `format_content` reads `draft`, sends it to the model with formatting instructions, and returns the result under `final_content`.

This pattern — read what you need, return what you changed — keeps each node focused on a single responsibility.

Building the Graph

We assemble the graph in three steps:

- Create a `StateGraph` instance by passing our state class. This gives us a builder object where we register everything.
- Register each node using `add_node("name", function)`, which associates a name with a function.
- Connect the nodes using `add_edge(source, target)` to define the execution order.

At this point, the graph structure is complete but not yet runnable.

Compiling and Running

`compile()` validates the graph and returns an executable pipeline. We run it with `invoke()`, passing an initial state dictionary that includes the topic and empty strings for the remaining fields.

The pipeline runs each node in order. The same compiled pipeline works with any topic — the structure is fixed, but the output adapts based on the input.

Instructions

1. Checkpoint 1 Enabled

Define the State Schema

Let's define the state structure that will monitor data as it moves through our content creation pipeline.

Create a PipelineState class that inherits from TypedDict with three string fields: topic, draft, and final_content.

2. Checkpoint 2 Step instruction is unavailable until previous steps are completed

Create the Node Functions

With our state schema in place, let's create two node functions that accept a state parameter of type PipelineState to handle content creation and formatting in our pipeline.

- **Drafting:** Define a function draft_content that uses the llm to generate a short blog post (2–3 paragraphs) based on the topic, and returns a dictionary with a draft key containing the generated content.
- **Formatting:** Define a function format_content that uses the llm to format the draft into a polished post with a title, section headings, and a brief conclusion, and returns a dictionary with a final_content key.

3. Checkpoint 3 Step instruction is unavailable until previous steps are completed

Build the Graph Structure

Now that both node functions are ready, let's construct the graph by adding them as nodes and defining the edges that connect them in order.

- Create a StateGraph instance with PipelineState as the state type and store it in builder.
- Add a draft node (using draft_content) and a format node (using format_content).
- Wire the execution flow by adding edges: START → draft → format → END.

4. Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Compile and Run the Pipeline

With all nodes and edges in place, let's convert our graph into an executable pipeline and test it with a sample topic.

- Call compile() on builder and assign the result to publishing_pipeline.

- Invoke it with an initial state dictionary containing the topic variable and empty strings for `draft` and `final_content`, storing the output in `result`.
- Uncomment the `print()` statements to display the topic, draft preview, and final content.
- Use the `invoke()` method on the `publishing_pipeline`, passing a dictionary with all three state fields as keys.
- Make sure to set both `draft` and `final_content` to empty strings in the initial state dictionary.

State Management

Our pipeline works, but each node writes to exactly one field, replacing what was there before. What if we need a field that expands as the graph runs — like an edit log tracking each node's actions?

Overwrite vs. Accumulate

By default, LangGraph *overwrites* a field when a node returns a new value for it. This works for fields like `draft`, where each node produces a fresh result. But some fields need to accumulate values across nodes — for example, an `edit_log` that records every step. This is where reducers come in.

A **reducer** controls how a field updates — combining the previous and new values instead of replacing. We attach one using Python's Annotated type hint. For list fields, a common reducer is `operator.add`, which concatenates lists, so returning `["new entry"]` from a node *appends* to the existing list.

Enriching the State Schema

We extend the pipeline's state with two new fields:

- `content_type` — a string specifying whether we are creating a blog post, social media post, or newsletter.
- `edit_log` — a list annotated with `operator.add`, where each node appends a short description of its actions.

Updating Multiple Fields at Once

A node can return more than one key. LangGraph applies the appropriate update method for each — overwriting standard fields and using the reducer for annotated ones. The draft

node, for example, returns both draft (overwritten) and edit_log (appended) in a single dictionary.

After both nodes run, the state contains the topic, content type, raw draft, final output, and a two-entry edit log — one from each node.

Driving Behavior with State

The content_type field guides how each node behaves. Setting it to “blog” creates a post with headings and a conclusion; “social_media” creates a short post with hashtags. The graph structure stays the same — only the output changes.

Running the pipeline twice with different content types also demonstrates the reducer at work: each run produces a two-entry edit log with entries appended, not overwritten.

Instructions

1. Checkpoint 1 Passed

1.

Define the State Schema with a Reducer

Let’s define a state structure that tracks data throughout the publishing workflow, including a record of edits.

- Create a PublishingState class that inherits from TypedDict with the following string fields: topic, content_type, draft, and final_content.
- Include an edit_log field that uses Annotated with a list type and the operator.add reducer to accumulate editing actions across the workflow.
- Make sure to include TypedDict in parentheses after the class name to inherit from it.
- Make sure to specify str as the type annotation for the topic, content_type, draft, and final_content fields.
- Use Annotated with a list type as the first argument and operator.add as the second argument for the edit_log field.

2. Checkpoint 2 Passed

2.

Create Nodes that Update Multiple Fields

With our state schema in place, let's create two node functions that accept a state parameter of type `PublishingState` to handle content creation and formatting.

- **Drafting:** Create a function `draft_node` that generates a short post (2–3 paragraphs) based on the topic and `content_type`, and returns a dictionary with keys `draft` (the generated content) and `edit_log`.
- **Formatting:** Create a function `format_node` that formats the content for publication based on the draft and `content_type`, and returns a dictionary with keys `final_content` (the formatted content) and `edit_log`.
- Use the `invoke()` method on the `llm` object, passing an f-string that includes the relevant state fields.
- Make sure to access the `content` attribute from the result object to get the generated text.
- Make sure to return a dictionary with both the `content` field and the `edit_log` field as a list containing a single string.

3. Checkpoint 3 Passed

3.

Build and Compile the Graph

Now that both node functions are ready, let's build the graph by adding them as nodes, defining the execution flow, and compiling it into a runnable pipeline.

- Create a `StateGraph` instance with `PublishingState` as its state type and assign it to `builder`.
- Add a draft node (using `draft_node`) and a format node (using `format_node`), then wire the execution flow by adding edges: `START → draft → format → END`.
- Call `compile()` on `builder` and assign the result to `pipeline`.
- Use the `add_node()` method with the node name as the first argument and the function reference as the second argument.
- Use the `add_edge()` method with the source node as the first argument and the destination node as the second argument.

- Make sure to use START and END constants (not strings) when defining edges to and from entry and exit points.

4. Checkpoint 4 Passed

4.

Run the Pipeline with Different Content Types

With our pipeline compiled, let's test it by generating content in two different formats using the same topic.

- Invoke the pipeline with a state dictionary setting topic to the topic variable, content_type to "blog", and draft, final_content, and edit_log to empty values. Store the result in result_blog.
- Invoke the pipeline again with content_type set to "social_media" (and the same empty values for the other fields). Store the result in result_social.
- Uncomment the print() statements to see the edit log and content previews for both formats.
- Use the invoke() method on the pipeline, passing a dictionary with all five state fields as keys.
- Make sure to set draft and final_content to empty strings in the initial state dictionary.
- Make sure to set edit_log to an empty list in the initial state dictionary.

Conditional Routing

Our pipeline currently follows a fixed path, with each input passing through the same sequence of nodes. When we have different content types, one function must handle all formats. Instead, we can let the graph select different paths based on the content of the state.

Conditional Edges and Routing Functions

A regular edge always leads to the same next node. A **conditional edge** operates differently — it first calls a **routing function**. This function receives the current state, checks a value (such as content_type), and returns the name of the node that should run next.

We use add_conditional_edges() to connect a source node to the routing function. We also specify the Literal type hint to declare which node names the function is permitted to

return — for example, `Literal["node A", "node B"]`. This makes the allowed paths explicit and helps catch errors early.

If the return values do not exactly match the registered node names, a path map can be provided as a third argument to `add_conditional_edges()` to translate between them.

Format-Specific Nodes

Instead of one formatting node with internal branching, we create two separate nodes — one for blog formatting (`format_blog`) and one for social media formatting (`format_social`). Each node focuses on a single output format, keeping the logic straightforward and easy to modify.

Both formatting nodes connect to `END`, so regardless of the chosen path, the pipeline terminates after formatting.

The graph looks like this:

Running Both Paths

Running the pipeline with `content_type` set to `"blog"` routes the draft through `format_blog`, creating a structured post with headings and a conclusion. Running it again with `"social_media"` sends the same draft through `format_social`, producing a condensed post with hashtags.

The graph structure remains consistent across each run. The routing function looks at the state and selects the appropriate path. This approach makes the pipeline flexible, without making the nodes more complex.

Instructions

1. Checkpoint 1 Passed

1.

Define Format-Specific Node Functions

Let's create two specialized formatting functions that accept a state parameter of type `PublishingState` and handle different content output formats.

- **Blog:** Define a function `format_blog` that formats the draft as a blog post (with title, section headings, and conclusion), and returns a dictionary with `final_content` (the formatted post) and `edit_log`.

- **Social:** Define a function `format_social` that condenses the draft into a social media post (<300 characters) with 3–5 relevant hashtags, and returns a dictionary with `final_content` (the formatted post) and `edit_log`.
- Use the `invoke()` method on the `llm` object, passing an f-string that includes the draft from the state.
- Make sure to access the `content` attribute from the result object to get the generated text.
- Make sure to return a dictionary with both `final_content` and `edit_log` fields, with `edit_log` as a list containing a single string.

2. Checkpoint 2 Passed

2.

Define the Routing Function

Now let's create a routing function that decides which formatting node to use, based on the content type.

- Create a function `route_by_content_type` that takes a state of type `PublishingState` and returns either `"format_blog"` or `"format_social"` using a `Literal` type.
- If the `content_type` in the state is `"social_media"`, return `"format_social"`; otherwise, return `"format_blog"`.

3. Checkpoint 3 Passed

3.

Build the Graph with Conditional Edges

With all node functions and routing logic ready, let's build the graph by adding nodes and defining the execution flow with conditional routing.

- Create a `StateGraph` instance with `PublishingState` as its state type and assign it to `builder`.
- Add three nodes: `draft` (using `draft_node`), `format_blog`, and `format_social`.
- Wire the execution flow with an edge from `START` to `draft`, a conditional edge from `draft` using `route_by_content_type`, and edges from `format_blog` and `format_social` to `END`.

- Use the `add_node()` method with the node name as the first argument and the function reference as the second argument.
- Use the `add_edge()` method with the source node as the first argument and the destination node as the second argument.
- Use the `add_conditional_edges()` method with the source node as the first argument and the routing function reference as the second argument.

4. Checkpoint 4 Passed

4.

Compile and Test the Router

With the graph fully wired, let's compile it and test both content types to see conditional routing in action.

- Call `compile()` on `builder` and assign the result to `routing_pipeline`.
- A topic variable and a loop are already provided — within the loop, invoke `routing_pipeline` with a state dictionary containing `topic`, the loop variable `ct` as `content_type`, and empty values for `draft`, `final_content`, and `edit_log`.
- Store the output in `result`.
- Use the `invoke()` method on the `routing_pipeline`, passing a dictionary with all five state fields as keys.
- Make sure to set `draft` and `final_content` to empty strings in the initial state dictionary.
- Make sure to set `edit_log` to an empty list in the initial state dictionary.

Cycles & Iteration

Conditional routing lets a graph choose between paths, but each path still runs once and ends. What if the output is not good enough on the first try? This is where **cycles** come in — edges that loop back to an earlier node, allowing a section of the graph to repeat.

How the Cycle Works

In our pipeline, the `revise` node connects back to `evaluate`, creating a loop: the evaluator scores the draft, the reviser improves it, and the evaluator scores it again. This repeats until the draft meets a quality threshold or a maximum iteration count is reached.

The Two-Model Pattern

We use two language models — a lighter model (gpt-4o-mini) for writing and revising, and a stronger model (gpt-4o) for evaluation. Because the evaluator is more capable than the writer, it produces genuine quality differences that drive the revision loop naturally. If the same model wrote and evaluated, it would likely rate its own output highly, and the loop would rarely trigger.

Evaluate, Route, Revise

The pipeline has three nodes:

- **draft** writes the initial blog post and sets the iteration counter to 1.
- **evaluate** sends the draft to the evaluator, which returns a quality score (1–10) and feedback.
- **revise** uses the feedback to produce an improved draft, incrementing the counter.

The routing function checks `quality_score` and `iteration` — returning "revise" to continue or "end" to stop. The third argument to `add_conditional_edges()` is a **path map** — a dictionary that translates these return values to actual targets (the "revise" node and END).

Preventing Infinite Loops

Cycles introduce the risk of infinite loops. LangGraph provides `recursion_limit`, passed to `invoke()` inside the config dictionary, which caps total node executions.

In practice, the routing function's own conditions (quality threshold and max iterations) are the primary safeguard; `recursion_limit` acts as a secondary safety net.

This config dictionary is also used for other runtime settings we will encounter later.

Instructions

1. Checkpoint 1 Passed

1.

Define the Editorial State and Draft Node

Let's define the state schema for our iterative editing pipeline and create an initial draft node that accepts a state parameter of type `EditorialState`.

- Create an EditorialState class that inherits from TypedDict with string fields topic, draft, and evaluation, and integer fields quality_score and iteration.
- **Drafting:** Define a function draft_node that uses llm_writer to create a rough blog post (2–3 paragraphs) based on the topic, and returns a dictionary with draft set to the generated content and iteration set to 1.
- Make sure to include TypedDict in parentheses after the class name to inherit from it.
- Use the invoke() method on the llm_writer object, passing an f-string that includes the topic from the state.
- Make sure to access the content attribute from the result object to get the generated text.

2. Checkpoint 2 Passed

2.

Define the Evaluate and Revise Nodes

With the state and draft node in place, let's create two functions that accept a state of type EditorialState to evaluate draft quality and revise it based on feedback.

- **Evaluating:** Define a function evaluate_draft that uses llm_evaluator to assign a quality score (1–10) and provide actionable feedback on the draft, then calls parse_evaluation_response() to extract score and feedback. Return a dictionary with evaluation (the feedback) and quality_score (the score).
- **Revising:** Define a function revise_draft that uses llm_writer to revise the draft based on the evaluation, increments the current iteration by 1 into new_iter, and returns a dictionary with draft (the revised content) and iteration (the updated count).
- Use the invoke() method on the llm_evaluator and llm_writer objects, passing an f-string that includes the relevant state fields.
- Make sure to access the content attribute from the result object to get the generated text.
- Use the parse_evaluation_response function with the result content as an argument to extract score and feedback.
- Use the addition operator (+) to increment the iteration value by 1 in the revise_draft function.

3. Checkpoint 3 Passed

3.

Define the Routing Function

Now that both the evaluate and revise nodes are ready, let's create a routing function that decides whether to continue revising or end the loop.

- Define a function `should_revise` that takes a state parameter of type `EditorialState` and returns either "revise" or "end" using a `Literal` type.
- If `quality_score` is below 8 and iteration is under 3, print the score and return "revise".
- Otherwise, store the reason for stopping ("quality threshold met" if the score is 8 or higher, else "max iterations reached") in `reason` and return "end".
- Make sure to include a type annotation for the state parameter and a `Literal` return type with the two possible string values.
- Use the `and` operator to check both the `quality_score` and iteration together in the if statement.
- Use a ternary conditional expression (if-else in a single line) to assign the appropriate string to the `reason` variable.

4. Checkpoint 4 Passed

4.

Building the Graph with a Cycle

With all nodes and routing logic ready, let's construct the graph with a cycle between evaluation and revision.

- Create a `StateGraph` instance with `EditorialState` as its state type and assign it to `builder`.
- Add three nodes: `draft` (using `draft_node`), `evaluate` (using `evaluate_draft`), and `revise` (using `revise_draft`).
- Wire the execution flow with an edge from `START` to `draft`, an edge from `draft` to `evaluate`, a conditional edge from `evaluate` using `should_revise` (mapping "revise" to `revise` and "end" to `END`), and an edge from `revise` back to `evaluate` to create the cycle.
- Use the `add_node()` method with the node name as the first argument and the function reference as the second argument.

- Use the `add_edge()` method with the source node as the first argument and the destination node as the second argument.
- Use the `add_conditional_edges()` method with the source node, routing function, and a path map dictionary.

5. Checkpoint 5 Passed

5.

Compile and Run the Pipeline

Let's compile the pipeline and run it with a sample topic to test the iterative editing process.

- Call `compile()` on `builder` and assign the result to `editorial_pipeline`.
- Invoke it with a state dictionary containing the topic variable, empty strings for draft and evaluation, 0 for `quality_score` and `iteration`, and a config parameter with `recursion_limit` set to 10. Store the output in `result`.
- Uncomment the `print()` statements to view the final score, iteration count, and draft preview.
- Use the `invoke()` method on the `editorial_pipeline`, passing a dictionary with all five state fields as keys.
- Make sure to pass the config parameter as a keyword argument with a dictionary containing `recursion_limit` set to the specified value.

Human-in-the-Loop

Our pipeline can now draft, route, and iteratively refine content — all without human involvement. But in a real publishing workflow, an editor needs to review the draft before it goes live. The graph should be able to pause, present the draft, and resume based on the editor's decision.

Checkpointing with MemorySaver

For a graph to pause and resume, its state must be saved. `MemorySaver` is an in-memory checkpointer — we pass it to `compile()` using the `checkpointer` parameter, and `LangGraph` saves the state after every node so the graph can pick up where it left off.

Each run is identified by a `thread_id` inside the config dictionary, keeping separate workflows isolated.

Pausing Execution

Inside the review node, we call `interrupt()` with a **payload** dictionary containing the draft and review instructions. The graph stops immediately, and the caller receives this payload. At this point, `interrupt()` has not returned a value to the node — execution is frozen on that line.

Resuming and Routing

The caller invokes the graph again with `Command(resume=...)`, passing the editor's decision. This input becomes the return value of the same `interrupt()` call, and the node continues from where it paused.

The review node parses the decision and routes with `Command(goto=...)` to publish if approved, or back to revise if changes are needed. Both accept an `update` parameter to modify state fields at the same time (for example, passing feedback so revise can use it).

Since `revise` connects back to `review`, the editor can request multiple rounds of changes before approving.

Execution Flow

- On the first invoke, the graph runs `write`, then `review` calls `interrupt()` and pauses. We inspect the payload to view the draft.
- We then resume with `Command(resume="revise: Add examples")`. The review node updates feedback in the state and routes to `revise`, which rewrites the draft and loops back to `review`, where the graph pauses again.
- Finally, we resume with `Command(resume="approve")`. The review node routes to `publish`, the status is set to published, and the graph reaches `END`.

Instructions

1. Checkpoint 1 Passed

1.

Define the Draft and Revise Nodes

Let's create two functions that accept a state parameter of type ReviewState to generate the initial draft and revise it based on human feedback.

- **Drafting:** Define a function write_draft that uses the llm to write a short blog post (2–3 paragraphs) based on the topic, and returns a dictionary with a draft key containing the generated content.
- **Revising:** Define a function revise_content that uses the llm to revise the draft based on the feedback, and returns a dictionary with a draft key containing the revised content.
- Use the invoke() method on the llm object, passing an f-string that includes the relevant state fields.
- Make sure to access the content attribute from the result object to get the generated text.

2. Checkpoint 2 Passed

2.

Define the Editor Review Node

With the draft and revise nodes in place, let's set up memory persistence and complete the function that pauses execution for human review.

- Create a MemorySaver instance and assign it to cp.
- **Reviewing:** Complete the editor_review function.
 - Use interrupt() to pause execution by passing a dictionary with the draft and an instruction to reply with "approve" or "revise: <feedback>". Store the response in decision.
 - If the decision starts with "approve", return a Command with goto set to "publish"; otherwise, return a Command with goto set to "revise" and update containing the extracted feedback.
- Make sure to include a return type annotation using Command with Literal specifying "publish" and "revise".
- Use the interrupt function, passing a dictionary with both draft and instruction keys.
- Make sure to set update to a dictionary containing the extracted feedback.

3. Checkpoint 3 Passed

3.

Build the Graph Structure

With all node functions ready, let's construct the graph by adding them as nodes and defining the execution flow.

- Create a StateGraph instance with ReviewState as its state type and assign it to builder.
- Add four nodes: write (using write_draft), review (using editor_review), publish (using publish_content), and revise (using revise_content).
- Wire the execution flow with an edge from START to write, edges from write and revise to review, and an edge from publish to END.
- Use the add_node() method with the node name as the first argument and the function reference as the second argument.
- Use the add_edge() method with the source node as the first argument and the destination node as the second argument.
- Make sure to use START and END constants (not strings) when defining edges to and from entry and exit points.

4. Checkpoint 4 Passed

4.

Compile and Run the Graph with Interruption

Now let's compile the graph with memory persistence and run it until it pauses for human review.

- Call compile() on builder with checkpointer set to cp, and assign the result to editorial_graph.
- Invoke it with a state dictionary containing the topic variable, empty strings for draft, feedback, and status, along with the config dictionary. Store the output in result.
- Uncomment the print() statements to view the draft and review instructions when the graph pauses.
- Make sure to pass checkpointer as a keyword argument when calling the compile() method on the builder object.

- Use the `invoke()` method on the `editorial_graph`, passing a dictionary with all four state fields as keys.
- Make sure to pass `config` as a keyword argument with the value set to the `config` variable.
- Make sure to set `draft`, `feedback`, and `status` to empty strings in the initial state dictionary.

5. Checkpoint 5 Passed

5.

Resume the Graph with Human Feedback

Now let's resume the graph by providing editor feedback for revision and then approving the final draft.

- **Revise:** Invoke `editorial_graph` with a `Command` whose `resume` is set to "revise: Add examples of where AI can safely reduce repetitive work", along with the `config` parameter. Store the result in `revised`.
- **Approve:** Invoke `editorial_graph` again with a `Command` whose `resume` is set to "approve", along with the `config` parameter. Store the result in `final`.
- Uncomment the `print()` statements to view the revised draft and the final status after approval.
- Use the `invoke()` method on the `editorial_graph`, passing a `Command` object as the first argument.
- Make sure to pass `resume` as a keyword argument when creating the `Command` object.
- Make sure to pass `config` as a keyword argument to the `invoke()` method.

Agent with Graph Memory

We have built pipelines that draft, route, loop, and pause for human review. But every run starts fresh, so the graph has no memory of past interactions. What if we want an assistant that remembers earlier messages, uses tools when needed, and keeps context across multiple turns?

Managing Conversation History

Earlier, we defined custom TypedDict schemas with fields like topic, draft, and edit_log. Conversational agents need a different structure — a growing list of messages.

MessagesState is a built-in state schema with a messages field that uses the add_messages reducer, so each new message is appended rather than replaced. The list stores objects like HumanMessage (user input) and AIMessage (model responses), preserving the full conversation across turns.

Binding Tools to the LLM

The assistant needs access to tools, such as count_words and save_drafts. We define tools using the @tool decorator, then call bind_tools() on the LLM to attach them.

Example

```
from langchain.tools import tool

@tool
def to_uppercase(text: str) -> str:
    return text.upper()

llm_with_tools = llm.bind_tools([to_uppercase])
```

When the model decides a tool is needed, it returns a message with tool_calls instead of plain text. The graph uses this to determine what happens next.

ToolNode and the Agent Loop

ToolNode is a pre-built node that automatically executes tool calls from the last message and adds results back to the history. The graph follows a two-node cycle:

- assistant calls the LLM with the full message history. If the response includes tool calls, routing sends execution to tools. Otherwise, the graph ends.
- tools runs the tool calls and loops back to assistant, which can then respond using the tool results.

Multi-Turn Memory

By compiling with MemorySaver and reusing the same thread_id, the message history persists between turns. The assistant can reference the user's name, earlier topics, or previously generated content. Starting a new thread_id creates a fresh session.

Bringing It All Together

This exercise ties together MessagesState (accumulation), conditional routing (assistant ↔ tools cycle), and MemorySaver (persistence) into a conversational agent built entirely as a LangGraph workflow.

Instructions

1. Checkpoint 1 Passed

Define the Tools and Draft Store

Let's set up the utility tools and storage that our publishing assistant will use during conversations.

- Create an empty dictionary called drafts_store to hold saved drafts. Then define two tools using the @tool decorator.
- Define a function count_words that takes a text parameter, splits it into words, and returns a formatted string showing the word count.
- Define a function save_draft that takes title and content parameters, saves content to drafts_store under the title key, and returns a formatted string showing the total number of drafts.
- Use the split() method on the text to separate it into words, then use len() to count them.
- Make sure to assign the content to the drafts_store dictionary using the title as the key.
- Use len() on the drafts_store dictionary to get the total number of saved drafts.

2. Checkpoint 2 Passed

Bind the Tools to the LLM

With our tools defined, let's register them with the language model so the assistant can call them during conversations.

- Create a list called `tools` containing the `count_words` and `save_draft` functions.
- Call `bind_tools()` on `llm`, passing in the `tools` list, and assign the result to `llm_with_tools`.
- Make sure to include the function references for both tools when creating the `tools` list.

3. Checkpoint 3 Passed

Define the Assistant Node and Routing Function

Now, let's create the assistant node and a routing function that accepts a state parameter of type `MessagesState` to process messages and decide whether to execute tools or end the turn.

- **Assistant:** Define a function `assistant_node` that invokes `llm_with_tools` with the state's messages, and returns a dictionary with a `messages` key containing the response as a list.
- **Routing:** Define a function `should_use_tools` that returns either `"tools"` or `"end"` using a `Literal` type.
 - Retrieve the latest message from the state's messages list and store it in `last_message`.
 - If `last_message` has a `tool_calls` attribute and it is non-empty, return `"tools"`; otherwise, return `"end"`.
- Use the `invoke()` method on the `llm_with_tools` object, passing the messages from the state.
- Make sure to wrap the response in a list when returning it in the messages dictionary.
- Use negative indexing to access the last message from the messages list.
- Use `hasattr` or `getattr` to safely check for or access the `tool_calls` attribute on `last_message` before using it.

4. Checkpoint 4 Passed

Build the Graph with a Tool-Calling Loop

With the assistant node and routing function ready, let's construct the graph that connects the assistant to its tools.

- Create a `StateGraph` instance with `MessagesState` as its state type and assign it to `builder`.

- Add two nodes: assistant (using `assistant_node`) and tools (using a `ToolNode` initialized with the tools list).
- Wire the execution flow with an edge from `START` to assistant, a conditional edge from assistant using `should_use_tools` (mapping "tools" to tools and "end" to `END`), and an edge from tools back to assistant to create the loop.
- Use the `add_node()` method with the node name as the first argument and the function reference as the second argument.
- Make sure to create a `ToolNode` instance with parentheses, passing the tools list as an argument when adding the tools node.
- Use the `add_edge()` method with the source node as the first argument and the destination node as the second argument.
- Use the `add_conditional_edges()` method with the source node, routing function, and a path map dictionary.

5. Checkpoint 5 Passed

Compile the Graph with Memory

Let's add persistent memory so the assistant can maintain conversation history, then compile the graph into a runnable pipeline.

- Create a `MemorySaver` instance and assign it to `memory`.
- Call `compile()` on `builder` with `checkpointer` set to `memory`, and assign the result to `publishing_assistant`.
- Make sure to pass `checkpointer` as a keyword argument when calling the `compile()` method on the `builder` object.

Recap

Across this module, we built a content publishing pipeline and progressively enhanced it — starting with a simple linear graph and ending with a conversational agent that reasons, acts, and remembers.

In this lesson, we:

- learned that `LangGraph` models workflows as directed graphs: nodes execute tasks, edges define flow, and a shared `TypedDict` state passes data between steps

- built the pipeline with StateGraph to define the state schema, add_node and add_edge to register steps and connections, and compile to produce a runnable graph
- controlled state updates with reducers — attaching operator.add through Annotated fields to accumulate values, while standard fields overwrite on each update
- added conditional routing with add_conditional_edges and Literal type hints, so the graph selects paths based on state values
- introduced cycles to loop between evaluation and revision until a draft meets a quality threshold or reaches a recursion_limit
- paused execution with interrupt for human review, resumed with Command(resume=...) to pass the editor's decision, and routed to the next step with Command(goto=...)
- created a conversational agent using MessagesState with its add_messages reducer to maintain message history
- attached tools to the LLM with bind_tools, executed calls via ToolNode, and persisted memory across turns using MemorySaver

Each exercise extended the same pipeline, showing how these capabilities layer on top of each other — from a fixed sequence to a looping, human-reviewed, memory-enabled workflow.

Congratulations!

Instructions

Examine the content publishing pipeline diagram. It shows how:

- a user request is parsed and routed either to a search tool (for external facts) or directly to drafting
- an LLM evaluates the draft and iterates through revisions until it meets quality standards
- a human reviewer approves or rejects the formatted output
- the memory store persists the published result and loads context for future requests

